GitHub 修复开发过程（脱密版）

整理日期：2026-10-19。以 10 月 1–15 日的开发记录为依据，参考此前 FlashInfer 文档的“现象—隔离—机制—补丁—验证—边界”结构。

Word 与 PDF

完整合集：Word · PDF。本目录每篇 Markdown 也提供同名 .docx 与 .pdf 分篇版。

导出版保留正文、表格与公开链接；流程图转为等义步骤列表。Word 可编辑，PDF 嵌入中文字体。可用仓库 tools/documents/export_github_repairs.py 重新生成，依赖 python-docx、reportlab、markdown、beautifulsoup4 及中文 TrueType 字体（通过 --font 指定）。

详细案例

案例	开发过程的重点	入口
Triton FP8 转换	舍入错误与不支持转换的分层定位；根据 review 改写 API 行为	详细记录
Quack autotuning	从全局 RNG 污染到独立 generator；状态和数值双重负对照	详细记录
DeepSpeed overflow	BF16 opt-out 修复引入 FP32 回归后的收窄与 sibling 路径验证	详细记录
Mooncake 队列	取消 reservation、唤醒、quota 与 FIFO 的状态机闭环	详细记录
CUTLASS workspace	从 Python 整数猜测收敛到 descriptor 的 Int32 extent 溢出	详细记录
vLLM padding	CPU 单测、主线合并、真实模型和 attention mask 对照	详细记录

先读修复流程与证据组织，再看具体案例。此前的FlashInfer 根因详解展示了源码、SASS、控制组和补丁 A/B 如何形成互相支持的证据。

文档范围

本次只整理开发过程，没有重新运行 GPU 测试或查询上游 PR 实时状态。测试数量和结果均注明所属阶段；公开链接用于定位原 PR，状态只代表原始记录日期。示意伪代码用于解释契约，不是可直接应用的补丁。

原始材料包括 10 月 1 日 Mooncake 取消排空记录、10 月 8 日 CPU review 回归、10 月 12–13 日三项目验证、10 月 14 日 Triton/Quack/DeepSpeed 修复以及 10 月 15 日 review 更新。这里移除了个人信息、内部地址、工作树绝对路径、运行标识、凭据上下文和部署拓扑；保留公开项目、源码相对位置、PR 链接、测试方法和已知限制。

归档与公开摘要之间的对应关系见近期工作总览。这组文档补充开发过程，不改变既有摘要所述的验收范围。

从故障到可审阅补丁：GitHub 修复流程

1. 以 FlashInfer 旧记录为参照

FlashInfer 案例最初表现为 RPC timeout，后续通过 TP、Graph、融合路径等控制变量，将范围缩到 fused kernel 的 polling。源码和 SASS 解释了 FTZ 如何影响 sentinel 判断，补丁前后 A/B 则证明了受测失败路径得到修复。

需要保留归属：位比较修复对应上游 PR，本地工作包括定位、回移、JIT 重建与回归，不能把整个上游修复归为个人原创。代表性触发位模式也不等于曾从原故障保存到具体 lane 的 payload。

近期 GitHub 修复沿用相同的证据组织方法：先说明哪个条件失败，再说明为什么失败，最后用同条件下的修复版测试说明改动的效果。

2. 固定问题与基线

先检查 issue 的触发条件、已有修复和 review 要求，确定本轮具体范围。固定源码版本，分别准备原版和修复版；新增测试可以同时放入两者，但要说明原版生产代码未改、测试文件已经扩展。

环境能导入不是验证成功。Triton 原生绑定必须匹配被测源码；FlashInfer 改 header 后还需要重建被实际加载的 JIT 产物；vLLM 复用 native artifacts 时要明确其版本限制。

3. 先证明失败发生在目标位置

案例	有效的失败证据	不能替代它的现象
Triton	目标转换触发特定断言或不支持错误	测试 masked load 自身构造错误
Quack	warmup 改变默认 CUDA RNG，后续样本不匹配	依赖安装失败
DeepSpeed	engine.step 的 skip 或参数状态违反目标精度契约	仅 hooks 通过
CUTLASS	descriptor 参数提取在边界 extent 处溢出	只看到“大 shape 失败”
vLLM	同 input IDs、匹配 mask 下的评分差异	不同 mask 的 HF 结果

先区分测试辅助脚本、依赖准备与生产代码的问题，避免把基础设施失败记成原 bug 复现。

4. 把补丁写成明确的契约

契约应说明哪些行为改变，哪些仍需成立。Quack 的目标是 warmup 不消耗应用 RNG；DeepSpeed 是仅在指定 BF16 opt-out 条件下放行，而不是关闭所有精度的保护；Mooncake 是回收取消槽位，但真实发送继续受 quota 控制。

对照用例覆盖正例、负例和相邻路径。除了“修复后通过”，还要确认原版在相同新测试下失败。源码机制解释与实测证据互相支持，不能相互替代。

5. Review 是实现的一部分

1. 固定失败条件和基线。

2. 在原版上复现目标失败。

3. 分析根因并实现最小补丁。

4. 用相同测试执行负正对照。

5. 提交可审阅差异及验证边界。

6. Review 若发现契约问题或回归，返回第 3 步；若需补证据，返回第 4 步。

7. 无新增修改时，等待维护者与 CI 流程。

Triton 的隐式 FP16 中转数值正确，但不符合维护者的接口设计；后续转向清晰拒绝并重新测试。DeepSpeed 初版通过部分精度测试，却在 FP32 上产生回归；需要修补并补全矩阵，不能只引用初版绿色结果。

保持已有 PR 上下文，以新增提交记录返修。主线冲突解决时保留双方测试，不能通过整文件覆盖丢弃其他功能。若上游要求人工审阅或贡献声明，记录实际完成的步骤，不把自动测试等同于人工认可。

6. 分层写验证结果

层级	应记录什么
编译检查	目标架构、预期拒绝/成功；不称为硬件运行
单元测试	新增与既有测试范围、原版失败原因；mock 的部分单独标注
集成测试	实际设备、后端、数值参考和配置范围
Smoke	证明启动或执行完成；未做数值参考时明确标注
CI	本次 head 的实际检查；skipped、waiting、未授权不记为 passed
Review/merge	批准、待复审、合入分别记；旧 diff 的批准不自动覆盖新改动

性能测试还需固定计时对象与统计方法。随机状态、数值错误或活性修复本身不自动构成性能提升。

7. 发布稿如何保留可复查性

每篇公开案例保留公开 PR、源码相对位置、触发条件、补丁思想、负正对照、实际覆盖范围与最后记录日期。内部日志路径、机器入口、账号和私有模型信息不进入稿件。

对未来复跑，按文中矩阵在固定公开源码版本上重建测试；不要把示意步骤当成已经随文交付的可执行脚本。原始实验未重跑时如实写“历史记录”，避免归档日期被误解为新验证日期。

Triton FP8 转换：从编译断言到明确的接口错误

开发记录：2026-10-14 至 10-15。公开入口：issue #12259、PR #12290。

1. 现象与最终选择

E4M3 与 E5M2 直接转换未指定舍入模式时，NVIDIA 下沉路径发生断言。最终修订选择在前端清晰拒绝不支持的直接转换，保留用户显式写出的 FP16 中转链和同格式转换。最初自动插入 FP16 的方案已被维护者要求撤回，不能继续把它描述成最终实现。

2. 定位：相同位宽不等于无需舍入

原前端默认舍入逻辑主要依据目标 primitive 位宽是否减小。两种 FP8 都占 8 bit，但指数和尾数分配不同，因此同位宽转换仍可能需要舍入。

第一次只补 RTNE 后，错误变为“不支持该格式对的转换”。这暴露出第二层限制：给出舍入模式不能凭空补齐后端转换实现。将两个错误分开，避免只修第一个断言就宣布问题解决。

涉及位置为 python/triton/language/semantic.py，回归加入已有 python/test/unit/language/test_core.py。

3. 第一版：隐式 FP16 中转

有限 FP8 值可以精确扩展到 FP16，再走已有 FP16 到 FP8 的舍入路径。第一版据此避免直接下沉到缺失转换。

原版与修复版共享同一份、匹配源码的原生编译库，只改变 Python 前端和新增测试，减少编译环境差异。测试曾因 masked load 使用整数填充值而在目标转换之前失败；改为浮点填充值后才进入有效的转换验证。

第一版记录中，SM89/SM90 双向编译检查通过，H200 新回归 6 项和既有转换子集 48 项通过。数值样本限制为共同范围的有限编码，覆盖 signed zero、次正规数和舍入边界；没有声称覆盖溢出及 NaN 策略。

4. Review：数值正确与语言契约是两件事

维护者不接受隐式 upcast/downcast。返修移除了自动中转，并验证三类行为：

输入操作	返修后的预期
直接 E4M3 → E5M2 或反向	清晰的前端错误，不进入底层断言
用户显式 FP8 → FP16 → 另一 FP8	保持可用并与独立参考比对
同格式 identity cast	保持全部编码

这一步不是继续扩展支持范围，而是明确当前能力边界。错误检查不能仅接受“任意非零退出”，否则内部崩溃也可能被误当作正确拒绝。

5. 返修验证

验证	结果	能证明什么
已发布旧补丁的编译对照	4/4 直接转换可编译	确认被移除的隐式行为原先存在
SM89/SM90 编译矩阵	12/12 通过	4 个前端拒绝、4 个显式链成功、4 个 identity 成功
H200 新回归	14 通过	含双向拒绝、6 个显式链数值检查、2 个 identity 检查
既有浮点转换子集	48 通过，110 未选入	已选范围没有发现回归
定向代码检查	12 通过，6 不适用	适用静态检查完成

数值链与 Torch 参考按字节比较；identity 覆盖全部 256 编码。SM89 没有物理运行验证，也未覆盖 AMD、解释器或完整项目 CI。

6. 发布与边界

10 月 15 日以普通新提交更新原 PR、正文及 issue 说明，并请求维护者复审；原记录仍为等待复审，不代表已批准或已合入。本次归档未查询最新状态。

本例的经验是先拆开 frontend 策略、lowering 能力和数值语义三个层次。可运行的替代实现仍须符合项目对隐式转换的设计要求。

Quack：autotuning warmup 污染应用随机序列

开发日期：2026-10-14；review 状态记录截至 10 月 15 日。issue #218 · PR #219。

1. 现象

调优 warmup 通过 torch.randn 生成矩阵，使用默认 CUDA generator。应用即使设定相同 seed，后续随机结果仍可能因本次调优是否冷启动而变化。问题不是矩阵乘算错，而是工具层消耗了应用的随机状态。

2. 原版最小复现

先对原版执行零时长 warmup，再比较 CPU 和 CUDA RNG 状态，并用独立克隆的 generator 生成后续参考样本。原版表现为 CPU 状态不变、默认 CUDA 状态改变、下一批随机值不一致。

零时长仍能暴露问题，说明污染发生在 warmup 准备阶段，不要求运行长时间调优循环。这个对照帮助把问题缩到随机矩阵初始化。

3. 补丁与不变量

生产改动位于 quack/autotuner.py：给现有随机矩阵生成传入独立、局部初始化的 CUDA generator。warmup 矩阵形状、BF16 类型、时长和乘法循环保持原逻辑。

示意契约：

```
应用默认 RNG ——应用随机操作——> 下一应用样本
独立 warmup RNG ——warmup 矩阵——> 调优预热
```

两个随机流分离，应用的下一样本不应受是否执行 warmup 影响。独立 generator 必须绑定目标设备，不能只在默认设备上验证。

4. 测试为什么同时看状态和值

新增测试放在已有 tests/test_autotuner.py，覆盖零/非零时长和默认/非默认 CUDA 设备。每次保存 CPU 及所有可见 CUDA RNG 状态；warmup 后先检查状态，再比较后续 64 个随机值与独立 generator 的输出。

状态检查证明调用没有推进共享 RNG，数值检查直接证明应用可观察结果保持一致。若只重复设置全局 seed 再比较，会掩盖 warmup 对状态的污染。

对照	原版	修复版
原 issue 的 warmup	CUDA 状态改变，后续样本不匹配	状态保留，样本逐位一致
新增 4 项回归	4/4 失败，每项 64/64 样本不匹配	4/4 通过
完整 autotuner 测试文件	未运行整文件	7/7 通过，无跳过
定向格式与 lint	不适用	Ruff check/format 通过

两侧使用相同源码基线与隔离 SDK 环境。SDK 依赖问题单独处理，不把依赖升级混入生产修复。

5. 发布后遇到的 CI 边界

10 月 14 日完成普通提交、推送、PR 和 issue 跟进。10 月 15 日记录显示，upstream checkout 在受信任的 pull_request_target 上下文拒绝 fork 代码，因此 Ruff 和后续 GPU 检查没有执行完成。

这属于工作流信任边界，不构成 RNG 修复失败的证据，也不能宣称 CI 通过。已向维护者说明，未添加不安全 checkout 绕过项。本次文档不重新查询当前 CI。

6. 结论范围

已证明受测 warmup 不污染应用随机状态。没有运行完整 CuTe kernel 套件，没有测性能收益，也没有据此承诺整个应用训练可完全确定。局部工具的无副作用契约应独立测试。

DeepSpeed：BF16 overflow opt-out 的修复与返修

开发记录：2026-10-14，审阅状态快照截至 10 月 15 日。issue #8805 · PR #8818。

1. 原问题与期望行为

BF16 配置显式关闭 overflow 检查后，ZeRO-1/2 的无效全局范数分支仍可能跳过更新。修复需要尊重已有 opt-out 契约，同时保持默认保护与其他精度的行为。

“允许 opt-out”并不是建议在训练中关闭保护；只是在用户明确选择该模式时，更新逻辑应与配置语义一致。

2. 初版为什么没有发现问题

初版修改范数检查 gate，并通过 BF16 正负对照、既有 FP16 测试和 H200 的 BF16 验证。但初版 gate 的条件范围过宽，未覆盖 FP32 的保护行为。

后续 review 发现两个 P1：FP32 在 Inf/NaN 时错误继续更新、参数被污染；ZenFlow overlap override 还保留另一个不一致的无效范数分支。前一轮绿色测试不能覆盖这两个新发现。

3. 把失败样本变成返修矩阵

FP32 使用原生 CPU/gloo 两 rank 复现，分别覆盖 ZeRO 阶段、clipping 开关、有限梯度、Inf 和 NaN。初版公开补丁每 rank 为 4 pass / 8 fail；失败表现是本应 skip 的异常更新污染参数。

返修把 opt-out 收窄到“BF16 且显式关闭检查”，并在普通路径与 override 中同步实现。契约如下，示意不等同于完整生产代码：

```
发现无效全局范数
 ├── BF16 且显式 opt-out：遵循 opt-out 更新语义
 └── 其他情况：保留原保护/跳步行为
```

没有改默认值、范数算法或梯度裁剪公式，也没有用放宽误差容限隐藏问题。

4. 为什么要测 override

父类路径正确并不能保证覆写 step() 的 optimizer 路径正确。除了分支级测试，还运行真实 CPU optimizer-process，以检查实际 worker 流程中的行为。mock dispatch/cleanup 的单测与原生进程集成分别记录。

验证	负对照	返修后
新回归文件	原基线 3 fail / 17 pass	20 pass：4 个原生分布式 FP32 case，16 个分支单测
FP32 CPU/gloo	初版公开补丁每 rank 4 pass / 8 fail	每 rank 12/12，异常输入 skip、参数 finite
CPU optimizer-process	原 override 每 rank 16 pass / 8 fail	每 rank 24/24，8 个配置
H200/NCCL	原始未修复源码每 rank 28/36 步检查通过	每 rank 36/36，12 个配置
既有 overflow suite	不以旧结果替代返修回归	36 pass

不同负对照有意使用原基线或初版公开补丁：前者复现原 bug，后者复现修复引入的回归。它们不能混成一个“修复前”数字。

有限梯度更新还与独立 FP32 Torch SGD 参考在转换到目标 dtype 后逐位比较。这样可以同时确认异常 gate 与正常更新没有相互干扰。

5. 发布和范围

返修以普通新提交更新原 PR，并回复两条 P1、更新测试说明。10 月 15 日快照为已批准但未合入；跳过的 CI 不作为已执行测试，本次不重新查询状态。

原生 CPU worker 仅覆盖 full-update warmup，不含 selective/top-k；GPU 测试不含 optimizer offload、GPU ZenFlow 或 ZeRO3。没有完整模型收敛、吞吐或跨厂商验证。

这次返修说明，条件开关需要按 dtype、默认值和 override 路径建立测试矩阵。一个局部条件只有一行，也可能扩大实际生效范围。

Mooncake：取消 reservation 的排空与 worker 生命周期

开发记录：2026-10-01、10-08、10-12 至 10-13。PR #3661。

1. 现象：没有在途请求，队列仍会被耗尽

批量 admission 失败时，已经预留的位置不能直接回退共享 tail，否则可能破坏其他生产者的进度。实现将这些位置发布为空取消条目，让单消费者按 FIFO 回收。

取消条目不增加真实 inflight_slices。idle worker 只依据 inflight 工作时可能继续睡眠，取消槽位得不到回收，连续拒绝最终耗尽队列。这是资源回收与唤醒状态机的问题。

2. 第一层修复仍不足：唤醒不等于排空

最初补丁加入 publication readiness、取消唤醒和等待 predicate。但发送额度检查位于消费之前时，quota 不足仍会跳过取消条目。消费者即使已经醒来，也不能回收槽位。

最终顺序是：发布空条目 → 唤醒 idle worker → readiness 允许进入消费路径 → 在发送 quota 检查前清理队首取消条目 → 真实请求继续接受原 quota/priority 控制。

3. 并发契约

部分	必须保持的约束
Readiness	检查当前 head 的 sequence；后面的已发布位置不能越过未发布队首
Publication	release 发布与 acquire 读取配对，先确认就绪再读取 payload
空条目清理	仅单消费者执行，复用已有 pop 回收；遇到真实或未发布 head 立即停止
Quota	空条目回收不需要发送额度，真实请求和 overflow 仍受控制
唤醒	发布后在对应 mutex 下协调 suspend/notify，等待 predicate 使用同一协议
Accounting	不把取消条目伪装成真实 inflight 请求

head 的 relaxed 读取依赖 MPSC 单消费者条件，不能把这一方案直接用于多消费者队列。

4. 定向验证

新增测试覆盖队首发布顺序、连续取消超过容量、零 inflight 唤醒、无 quota 排空、真实队首停止、未发布队首停止和槽位全部返还。

验证	结果	边界
真实队列测试	13 项通过	6 既有、7 新增
重复运行	100 轮、1300 次通过	重复成功不等于形式化并发证明
旧 quota 顺序模型	8 次取消后槽满	控制流模型，不是完整旧 worker 集成
新顺序模型	32 次取消完成，真实发送为 0	证明取消回收没有借机绕过真实发送 quota

这一阶段没有验证真实网卡、多节点 RDMA、TSAN 或完整 transport 套件。

5. 后续 CI 归因：区分构建环境与队列逻辑

后续还处理了 wheel 与安装 Torch 版本不匹配的问题。缺少目标版本的扩展模块不能直接解释为队列代码回归。10 月 8 日的 selector 单测 8 项通过，但没有把较早的 wheel/collective 测试算作新提交重跑结果。

主线合并时只处理实际冲突，保留原修复与新增上游逻辑；没有借环境修复扩展修改队列算法。

6. metadata lifetime fixture 修复

后续原生测试在 admission 处失败，尚未进入预期 metadata lifetime 检查。原因是测试只创建合成 worker contexts，没有建立生产 guard 所要求的 running 状态。

补丁仅在 fixture setup/teardown 设置对应生命周期状态，没有放松生产 admission guard 或删除断言。原失败方法在相同条件下通过；相关 RDMA 测试为 105 pass、1 skip，队列测试 13/13。

该 fixture 修复于 10 月 13 日记录为已推送原 PR；仍不表示真实 RNIC 或完整 CI 通过。本次归档不重新查询 review/merge 状态。

7. 经验

并发故障需要追踪预留、发布、可见性、唤醒、额度和回收的整个状态转换。修复后测试失败也应先确认失败层级：环境不匹配、fixture 缺状态与生产算法错误是不同问题，需要不同证据。

CUTLASS：大 workspace 的 descriptor 溢出定位

开发记录：2026-10-12 至 10-13。PR #3614。

1. 现象与修正后的根因

FMHA backward 在大 workspace 场景失败，最初关注 Python 侧字节数算术。但 host int() 转换不足以修复问题：真正失败的是动态 DLPack descriptor 的 Int32 extent，在 byte workspace 达到 2**31 时溢出。

因此需要区分三步：Python 计算大小、CUDA storage 分配、descriptor 提取传参。大尺寸失败本身不能证明显存不足或 kernel 索引错误。

2. 缩小复现范围

在实际 DSL 环境中单独创建 workspace descriptor，并调用 .__c_pointers__()。动态描述在边界及以上失败，静态描述可工作，进而将原因缩到 descriptor 表示而非整个 Attention 数学计算。

测试三个字节长度：2147483647、2147483648、5697536000。前两项贴住有符号 32 位边界，第三项覆盖原大 workspace 级别。这里的数字是公开库边界测试参数，不是私有模型配置。

3. 补丁选择

修改位于 examples/python/CuTeDSL/cute/blackwell/kernel/attention/fmha/fmha_bwd.py，并新增对应 workspace 回归文件。新的 factory 使用静态 byte workspace descriptor，同时保持底层 CUDA uint8 storage 生命周期。

kernel 只需要 byte pointer，原 workspace offset 逻辑保持不变。保留 host 整数算术处理，但不再把它单独描述为完整修复。descriptor 与 storage 生命周期一起检查，避免“参数可转换”却留下悬空存储。

4. 验证分三层

层级	实际结果	不能据此声称
Descriptor 边界回归	B200 / DSL 4.8.0 下 3/3 通过	全部 Attention 形状正确
小形状完整 backward	数值 reference 通过	大形状同样经过 reference
大形状 workspace	约 5.70 GB launch smoke 通过	大形状数值已验证；该次跳过 reference

10 月 13 日补丁发布沿用前一日完成的 B200 证据，没有声称当天重新跑了 GPU。静态检查使用定向 fatal-error 项，新测试格式检查通过；没有把 kernel 全文件格式检查写成通过，也没有为了格式化重排无关实现。

5. Review 与经验

使用普通新提交更新原 PR，并更正根因与测试范围，请求 reviewer 复审。旧 diff 的批准不覆盖新 factory 和测试，本次文档不宣称当前已合入。

该案例的关键是把数据表示边界从计算边界中分离。Python 大整数、GPU storage 和外部 descriptor 的宽度规则不同，边界测试应落在发生窄化的实际层级。

vLLM：Cross-encoder padding 修复的模型级验证

开发记录：2026-10-08、10-12 至 10-13。PR #58143。

1. 问题与修复范围

Cross-encoder 评分中的 segment/token type 与 padding 边界需要与 tokenizer 语义一致。已有补丁采用文档区间的半开区间表示，保留 tokenizer 的 padding type，并增加非法 token type 回归。

本轮主要工作是主线同步和补齐 reviewer 要求的真实模型证据，不能把全部过程描述成再次新增生产实现。

2. 先保留主线和修复双方的测试

10 月 8 日合入主线时，手动冲突集中在 scoring 测试文件。保留本 PR 的区间/padding 回归，也保留上游新增 reranker、RequestError 等测试，未用整体 ours/theirs 覆盖。

实际源文件 CPU 单测 10 项通过，变更文件的定向检查通过。但 CPU 输入处理验证不能替代模型评分，因此继续准备 main/branch/HF 三方对照。

3. 环境失败不能算作模型验证

模型运行曾被 native attention 扩展缺失、依赖版本及辅助通信路径问题阻塞。这些失败先于目标评分逻辑，不构成修复正负对照。后续补齐隔离依赖并使用正式支持的 Attention 后端配置后才运行完整评分矩阵。

最终仍复用了已有 native artifacts，而不是完整构建当时主线的全部原生依赖。报告保留这一限制，没有将 source branch 对照等同于全新 native build 验收。

4. 关键控制变量是 mask

选择固定公开 BERT cross-encoder checkpoint，使用 H200、FP16、eager、单卡和未激活 raw logits。main/branch 使用相同 input IDs，分别比较无 padding 和右侧 padding。

受测 vLLM 路径将追加 PAD 作为可见位置。匹配的 HF padded 参考因此使用 all-ones attention mask，并使用 tokenizer 的 padding token type。另设 HF 默认 masked-padding 对照，避免混淆。

对照	目的
main vs branch，无 padding	确认正常路径不变
main vs branch，右侧 padding	观察修复触发路径
branch vs HF，匹配 mask	检查对应语义下的评分一致性
HF 默认 masked padding	证明 mask 的不同本身会改变参照，不作为 visible-padding 的等价目标

这项 PR 没有修改生产 attention-mask 语义，不能声称修复后自动等价于所有 HF padding 设置。

5. 实际结果

main/branch 共完成 12 组评分，另有 HF 参考与 mask control。无 padding 的 main/branch 最大绝对差为 0；branch 与 HF 的最大绝对差约为 0.001401。

右侧 padding 时，相对匹配 mask 的 HF 参考，main 最大绝对差约 5.286148，branch 约 0.001151。这里是实际观测值，没有为得到通过结果放宽已有断言；也没有把不同 mask 下的 HF logits 混入同一比较。

6. 发布与经验

10 月 13 日更新原 PR 的验证说明和 reviewer 回复，本轮没有新增 vLLM 生产代码提交。记录为等待复审，本次不查询实时合入状态。

本例说明“同样 padding 长度”不代表输入语义一致。必须同时核对 token IDs、token types、attention mask、精度与输出激活方式，模型级证据才可解释。